Air Quality Intelligence System

Advanced Database Management Systems

Péter Dobosi (DOP3BP)

University of Pannonia

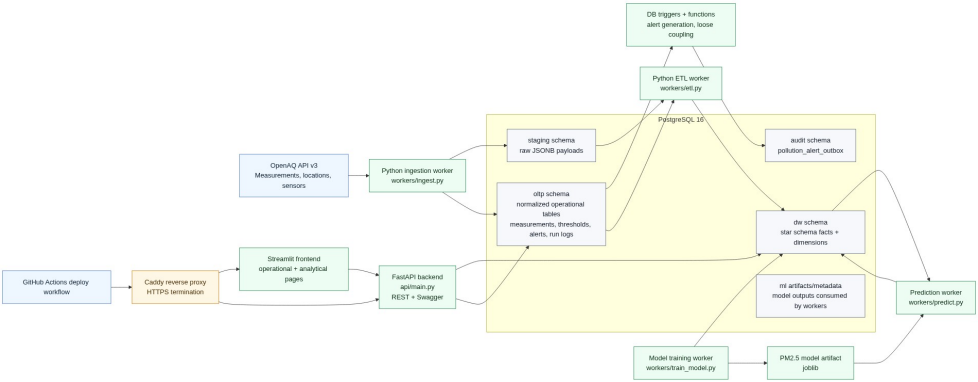
2025/26 Spring

Problem Statement

- Urban air pollution is a growing health concern
- Publicly available sensor data (OpenAQ) lacks real-time alerting
- Need: automated ingestion, threshold-based alerting, and operational oversight

Goal: Build a production-ready DBMS-backed monitoring system with triggers, transactions, and an operational GUI.

Architecture Overview



Technologies Used

- PostgreSQL 16
- PL/pgSQL triggers
- Range partitioning
- Explicit transactions
- CHECK / FK constraints

- Python 3.12
- FastAPI (REST)
- Streamlit (GUI)
- Docker Compose
- Caddy (HTTPS)
- GitHub Actions (CI/CD)

Demo: Operational GUI — Stations & Measurements

- Station Explorer page: select city → station → parameter
- Shows time-series chart and raw measurement table
- Data comes from `oltp.measurement_raw` (partitioned)

Live demo: open `https://mw79on-demo.online`, navigate to Station Explorer

Demo: Threshold Management

- CRUD interface for `oltp.threshold_rule`
- Each rule: city + pollutant + severity level + minimum value
- Unique constraint: `(parameter_id, city, warning_level)`

Live demo: add/edit a threshold rule, show it persisted in the table.

Demo: Trigger-Based Alert Generation

- Trigger: `trg_measurement_generate_pollution_alert`
- Fires AFTER INSERT on `measurement_raw`
- Matches value against active threshold rules $\rightarrow$ inserts alert row
- Second trigger enqueues event into `audit.pollution_alert_outbox`

Live demo: insert measurement $\rightarrow$ observe new alert appearing.

Demo: Alert Review & Transaction Safety

- Alerts page: filter by status / severity
- Transition: open $\rightarrow$ reviewed $\rightarrow$ closed (with notes)
- Ingestion uses explicit transactions with savepoints
- Partial batch failure $\rightarrow$ atomic rollback, no corrupt data

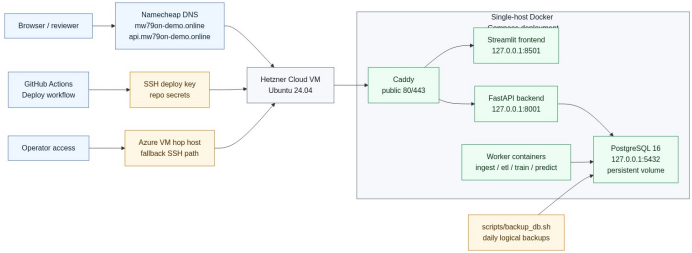
Live demo: review an alert; show ingestion log for transaction evidence.

Demo: Indexing, Partitioning, Backup

- `measurement_raw`: range-partitioned by month (25+ partitions)
- B-tree indexes on FKs and hot query columns
- `scripts/backup.sh`: `pg_dump` with rotation & optional off-site
- System Status page shows ingestion run history

Deployment & DevOps

- Hetzner Cloud VM (Ubuntu 24.04, 2 vCPU, 4 GB RAM)
- Docker Compose: DB + API + Frontend + Caddy
- Caddy handles automatic Let's Encrypt TLS
- GitHub Actions: push to `main` → rsync → docker compose up
- UFW firewall: only 22, 80, 443 open



Summary & Lessons Learned

- PostgreSQL triggers provide powerful event-driven logic
- Partitioning enables efficient time-range queries and archival
- Explicit transactions with savepoints ensure data integrity
- Docker Compose + Caddy = simple production deployment
- CI/CD keeps the system always up-to-date

Live system: `https://mw79on-demo.online`
Source code: `https://github.com/dobosipeter/advanced-data-warehouse-and-database-management-systems-submission`